

V = non terminals
 T = terminals
 P = production
 S = start

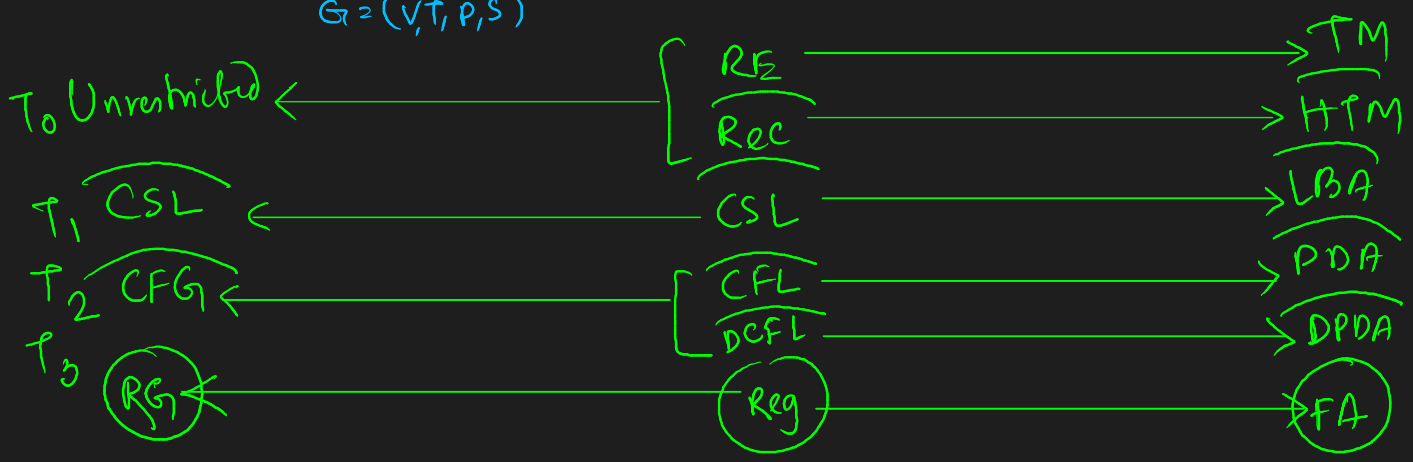
Revision

Set of Rules used to generate strings of a language
 $G = (V, T, P, S)$

set of strings formed from a alphabet.

Language

Abstract machine that reads i/p symbol & changes states acc to predefined rules.
Automata



FA → Mathematical model of computation used to recognize patterns and process strings.

DFA

- Deterministic
- For every state & input symbol there exists only 1 transition.
- $M = (Q, \Sigma, \delta, q_0, F)$
- $\delta : Q \times \Sigma \rightarrow Q$
- ϵ -moves not allowed.
- NO. of states $\uparrow$
- ϕ -state allowed

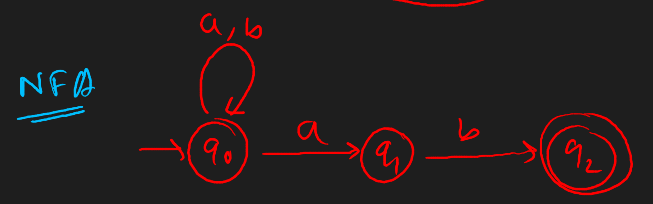
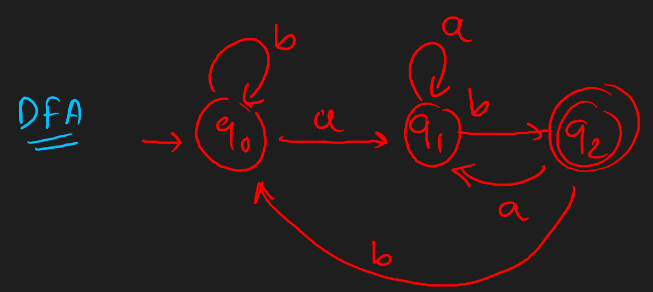
NFA

- Non deterministic
- For 1 input symbol can have any no. of ip symbols.
- $M = (Q, \Sigma, \delta, q_0, F)$
- $\delta : Q \times \Sigma \rightarrow 2^Q$
- ϵ -allowed. (ϵ -NFA)
- No. of states $\downarrow$
- ϕ state may or may not be allowed

Example: DFA & NFA

$\Sigma = \{a, b\}$

$L = \{w \in \{a, b\}^* \mid w \text{ ends with } ab\}$



$M = (Q, \Sigma, \Gamma, \delta, q_0, \bar{q}, F)$

initial stack symbol.

PDA

FA + Stack. $\delta(q, a, x)$

DPDA

- Deterministic
- Current state, i/p symbol, top symbol. → 1 move.

NPDA

- Non-det.
- Multiple possible moves.

Regular Language

- Accept by FA (DFA/NFA)
- $L = \{a^n \mid n \geq 0\}$ ✓
- $L = (a+b)^*$ ✓
- $L = \{a^n b^n \mid n \geq 0\}$ X

Regular Grammar

- Every production either Right/Left linear.
- $A \rightarrow aA$: right
- $A \rightarrow Aa$: left

Pumping Lemma → used to prove that a language is not Regular.

$w \in L, |w| \geq p \mid w = xyz \text{ where } |xy| \leq p, \text{ and } xy^iz \in L \forall i \geq 0$

→ if Regular.

DPDA

- No guessing
- Power ↓
- DCFL

NPDA

- can guess the correct path.
- Power ↑
- CFL

For FA $\rightarrow$ DFA $\equiv$ NFA

but PDA $\rightarrow$ DPDA $\leq$ NPDA

CFL

- Language generated by CFG.
- Accept by NPDA.

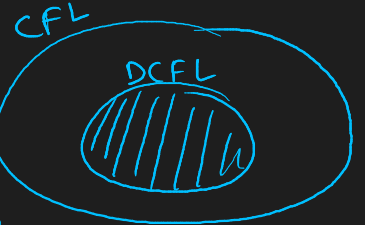
$a^n b^n$

CFG $\rightarrow$

- Grammar to generate CFL.
- $A \rightarrow \alpha$
 $\downarrow$
 Nonterm. $\rightarrow$ terminal / nonterm.

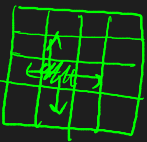
DCFL

A CFL that can be accepted by DPDA.



TM $\rightarrow$ Finite Control + ∞ tape

- ∞ tape
- read/write control
- head can move $\leftarrow, \rightarrow, \uparrow, \downarrow$



$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

$\Sigma \subseteq \Gamma$

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

states

tape alphabet

start state

blank symbol

$\delta(q, a) = (p, b, R)$

current state

Read

Write

Move Right

go to state p

Recursive lang

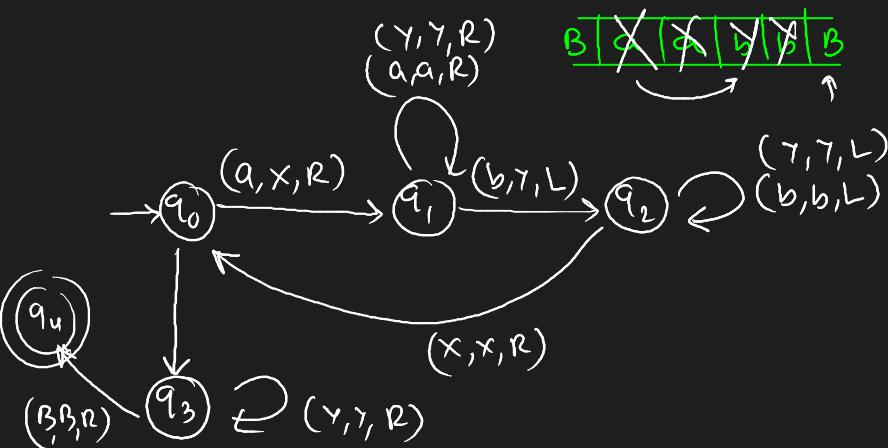
- decidable
- Exists a TM that accepts every string in L & rejects which are not in L
- Always Halts.
- Never Runs Forever.

LBA Linear Bounded Automata

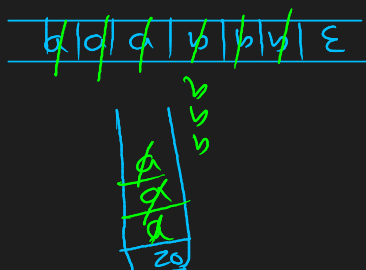
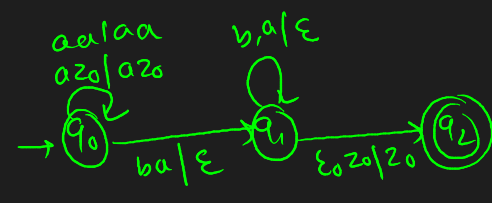
$\rightarrow$ TM with limited tape.

- head can move only inside the bounded region.
- Accepts CSL $\rightarrow$ language that can be generated by a CSG.

TM for $a^n b^n \mid n \geq 1$



PDA for $a^n b^n \mid n \geq 0$



RE (Recursively Enumerable)

- Turing Recognizable Lang.
- Accepts all strings in L
- Many never stop.

RE $\supset$ Recursive

UTM $\rightarrow$ TM that can simulate the operation of any other TM

UTM $(\langle M \rangle, w)$

Encoded description of machine M

RE $\rightarrow$ Regular Expressions

Formal Notation used to describe Regular Languages.

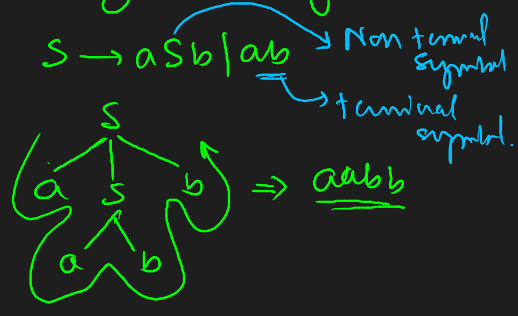
$a + b \rightarrow \{a, b\}$

$ab \rightarrow \{ab\}$

$a^* \rightarrow \{\epsilon, a, aa, \dots\}$

Parse Tree / Derivation tree.

↳ Tree representⁿ of derivation of string according to CFG



Mealy Machine

- Faster
- $\lambda: Q \times \Sigma \rightarrow \Delta$
- o/p based on current state & ip
- o/p attached to transⁿ
- # states $\downarrow$

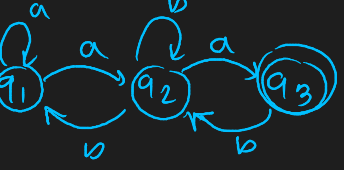
Moore Machine

- slower
- $\lambda: Q \rightarrow \Delta$
- o/p based on current state
- o/p attached to state.
- # states $\uparrow$

Arden's theorem

$R = Q + RP, \quad \epsilon \notin P,$
 then the unique solⁿ is $\Rightarrow R = QP^*$
 & if $R = Q + PR \Rightarrow R = P^*Q.$
 ↳ used to obtain a Regular Exp. from a FA.

NFA $\rightarrow$ RE



$$\begin{aligned}
 q_1 &= q_1a + q_2b + \epsilon \\
 q_2 &= q_2b + q_3a + q_1a \\
 q_3 &= q_2a \\
 \therefore q_2 &= q_2b + q_1a + (q_2a)b \\
 &= q_2b + q_1a + q_2ab \\
 &= q_1a + q_2(b+ab) \\
 \therefore q_2 &= q_1a + q_2(b+ab) \\
 \therefore q_2 &= q_1a(b+ab)^*
 \end{aligned}$$

$$\begin{aligned}
 q_1 &= q_1a + q_2b + \epsilon \\
 &= q_1a + q_1a(b+ab)^* + \epsilon \\
 &= q_1(a + a(ab+ab)^*b) + \epsilon \\
 &= \epsilon + q_1(a + a(ab+ab)^*b) \\
 \therefore q_1 &= (a + a(ab+ab)^*b)^* \\
 \therefore q_2 &= q_1a(b+ab)^* \\
 &= (a + a(ab+ab)^*b)^* a(b+ab)^* \\
 \therefore q_3 &= q_2a \\
 &= (a + a(ab+ab)^*b)^* a(b+ab)^* a
 \end{aligned}$$

Ans

Regular Grammar

- Type 3
- Recognized by FA
- Power $\downarrow$
- Prodⁿ:
 $A \rightarrow aB$
 or $A \rightarrow a$
- Generate Regular Languages

CFG

- Type 2
- PDA
- Power $\uparrow$
- $A \rightarrow \alpha$
- Generate CFLs.

Ambiguity

ambiguous if string generated by it has > 1 parse tree or has diff left/right most derivatⁿs.

Closure

A language class is closed under an opn if applying the opn still produces the same class.

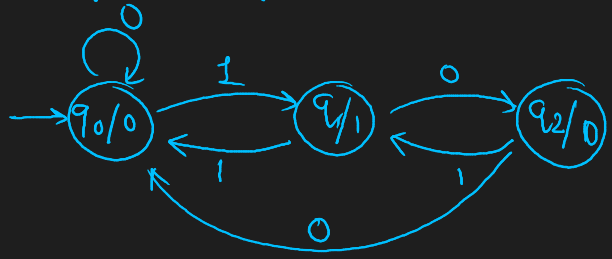
Kleene Star (*) & Kleene Plus (+)

$$\begin{aligned}
 L^* &= \bigcup_{i=0}^{\infty} L^i \\
 L &= (ab)^* \\
 &= \{ \epsilon, ab, abab, \dots \}
 \end{aligned}$$

$$\begin{aligned}
 L^+ &= \bigcup_{i=1}^{\infty} L^i \\
 L &= (ab)^+ \\
 &= \{ ab, abab, \dots \}
 \end{aligned}$$

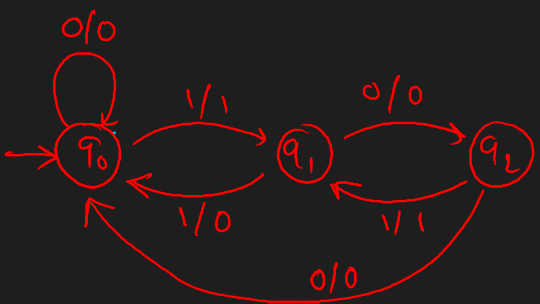
$$L^* = L^+ \cup \{ \epsilon \}$$

Moore $\rightarrow$ Mealy



CS	0	1	o/p
q ₀	q ₀	q ₁	0
q ₁	q ₂	q ₀	1
q ₂	q ₀	q ₁	0

Moore $\downarrow$ Mealy



CS	0	1
q ₀	q ₀ 0	q ₁ 1
q ₁	q ₂ 0	q ₀ 0
q ₂	q ₀ 0	q ₁ 1

Proof: $R = 0^*1(0+10^*1)^* = (1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1)$

$$\begin{aligned}
 0^* &= \epsilon + 00^* \\
 \therefore 0^*1 &= (\epsilon + 00^*)1 \\
 &= \epsilon 1 + 00^*1 \\
 &= 1 + 00^*1 \\
 \therefore R &= 0^*1(0+10^*1)^* \\
 &= (1+00^*1)(0+10^*1)^*
 \end{aligned}$$

Now, $A^* = \epsilon + A^*A$
 Let, $A = (0+10^*1)$
 $(0+10^*1)^* = \epsilon + (0+10^*1)^*(0+10^*1)$

Substitute into R.

$$\begin{aligned}
 \therefore R &= (1+00^*1) [\epsilon + (0+10^*1)^*(0+10^*1)] \\
 &= (1+00^*1) \epsilon + (1+00^*1)(0+10^*1)^*(0+10^*1) \\
 &= (1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1)
 \end{aligned}$$